

CHECKSTYLE

Tras hacer uso de la herramienta checkstyle de Maven (mvn checkstyle:checkstyle) para nuestro proyecto en relación al commit [62862b2](#) se ha inspeccionado el archivo generado checkstyle.html en target/site de nuestro backend y se han reportado estas clasificaciones de avisos:

- I. LeftCurly → 92 Violations
- II. NeedBraces → 6 Violations
- III. RightCurly → 92 Violations
- IV. RightCurly(2) → 92 Violations
- V. VariableDeclarationUsageDistance → 2 Violations
- VI. AvoidStarImport → 11 Violations
- VII. CustomImportOrder → 103 Violations
- VIII. Indentation → 918 Violations
- IX. MissingJavadocMethod → 45 Violations
- X. MissingJavadocType → 61 Violations
- XI. ModifierOrder → 8 Violations
- XII. LineLength → 68 Violations
- XIII. EmptyLineSeparator → 63 Violations
- XIV. FileTabCharacter → 656 Violations
- XV. WhitespaceAfter → 38 Violations
- XVI. WhitespaceAround → 141 Violations

- Reportados 2304 checkstyle violation , los cuales diferenciamos en varias categorias:

1.Estilo de formato e Identación (1968 avisos - ~85% del total):

- i. FileTabCharacter (656) e Indentation (918): Se deben al uso de tabuladores y sangría de 4 espacios habitual en el desarrollo con Java/Spring Boot, frente a los 2 espacios rígidos exigidos por la guía de estilo de Google.
- ii. WhitespaceAround (141), WhitespaceAfter (38) , LeftCurly (92), RightCurly (184) y EmptyLineSeparator (63): Falta o exceso de espacios en blanco alrededor de llaves, operadores y delimitadores.

2. Documentación de Javadoc (106 avisos):

- i. MissingJavadocType (61) y MissingJavadocMethod (45): Métodos y clases del modelo que carecen de bloques de comentarios de documentación.

3. Estructura e Importaciones (182 avisos):

- i. CustomImportOrder (103) y AvoidStarImport (11): Importaciones fuera del orden estándar o uso de comodines (*).
- ii. LineLength (68): Líneas que superan el límite de caracteres impuesto.
- iii. ModifierOrder (8) y NeedBraces (6): Modificadores desordenados o bloques if en una sola línea sin llaves.
- iv. VariableDeclarationUsageDistance (2): Distancia excesiva entre declaración de variable y su primer uso.

Justificación y decisiones del equipo:

- La inmensa mayoría de los avisos no representan un fallo de calidad crítica ni afectan a la ejecución o mantenibilidad básica del backend. Por este motivo, se considera que las violaciones de indentación y espacios son falsos positivos de estilo respecto a las reglas de Google. Se han abierto Issues específicas en el repositorio para abordar los avisos prioritarios de limpieza de código.

SPOTBUGS

Mediante el empleo de la herramienta `mvn compile spotbugs:check` se reportaron 31 bugs relacionados con patrones de código que pueden generar errores de seguridad, fugas de datos y/o bugs en producción.

Análisis Estático Funcional de Caja Blanca: SpotBugs:

Se clasifican los 31 avisos en 3 patrones de código y según su nivel de severidad (High, Medium, Low):

- Exposición de representación interna – Mal uso de mutabilidad (EI_EXPOSE_REP / EI_EXPOSE_REP2) – 26 bugs (Prioridad Media)
 - Ocurre cuando un getter o setter devuelve o asigna directamente un objeto mutable (como una Lista o Entidad JPA) sin hacer una copia defensiva (UnmodifiableList o clone). Esto permite que código externo modifique el estado interno de un objeto sin pasar por sus validaciones.
 - Afecta a:
 - Entidades JPA Purchase(user,session) y Session(movie,room).
 - Clase de servicio Block (items).
 - DTOs y Respuestas REST: ErrorsDto, AuthenticatedUserDto, BlockDto y MovieCatalogDto.
- Uso de codificación por defecto (DM_DEFAULT_ENCODING): 2 bugs (Prioridad Alta):
 - En JwtGeneratorImpl.java(líneas 28 y 37) se invoca String.getBytes() sin especificar el juego de caracteres (StandardCharsets.UTF_8).
 - Riesgo real: Si la app se despliega en un servidor cuyo SO tenga una codificación distinta, los token JWT generados o validados se corrompen y fallan los logins.
- Riesgos de Serialización y Excepciones de Seguridad (SE_BAD_FIELD, SE_NO_SERIALVERSIONID, REC_CATCH_EXCEPTION): 3 bugs (Prioridad Baja)
 - Se encuentra en la clase JwtFilter.java
 - Se hace una captura genérica de excepciones (catch (Exception e)) en lugar de excepciones específicas de JWT/Servlet, y falta de serialVersionUID o campos no serializables en un filtro que implementa Serializable.

JETM

MÉTRICAS OBTENIDAS DE RENDIMIENTO EN OPERACIONES DE SERVICIO

Measurement Point	#	Average	Min	Max	Total
CatalogService:findMovie	100	0,319	0,116	12,795	31,878

Measurement Point	#	Average	Min	Max	Total
UserService:login	100	67,824	64,820	111,181	6.782,419

Measurement Point	#	Average	Min	Max	Total
ShoppingService:buyTickets	100	4,221	2,605	26,516	422,073

La prueba mide el tiempo de respuesta en Integración (Servicio + BD) a lo largo de 100 ejecuciones continuas.

La diferencia de tiempos entre operaciones de búsqueda y autenticación reside en la seguridad y empleo de operaciones de coste computacional como hashing de la contraseña.

La diferencia con buyTickets radica en escritura transaccional (inserción en BD + actualización).

CONCLUSIÓN DE EVALUACIÓN NO FUNCIONAL

Los resultados confirman que el backend cumple de manera holgada con los umbrales de rendimiento necesarios para entorno de producción, garantizando fluidez en consultas grandes y solidez en transacciones de compras.